

SWITCH TRANSFORMERS: SCALING TO TRILLION PARAMETER MODELS WITH SIMPLE AND EFFICIENT SPARSITY

자연어처리팀: 박희수(발표자), 백지윤, 진명훈

Motivation

Consumption	CO ₂ e (lbs)
Air travel, 1 passenger, NY↔SF	1984
Human life, avg, 1 year	11,023
American life, avg, 1 year	36,156
Car, avg incl. fuel, 1 lifetime	126,000
Training one model (GPU)	
NLP pipeline (parsing, SRL)	39
w/ tuning & experimentation	78,468
Transformer (big)	192
w/ neural architecture search	626,155

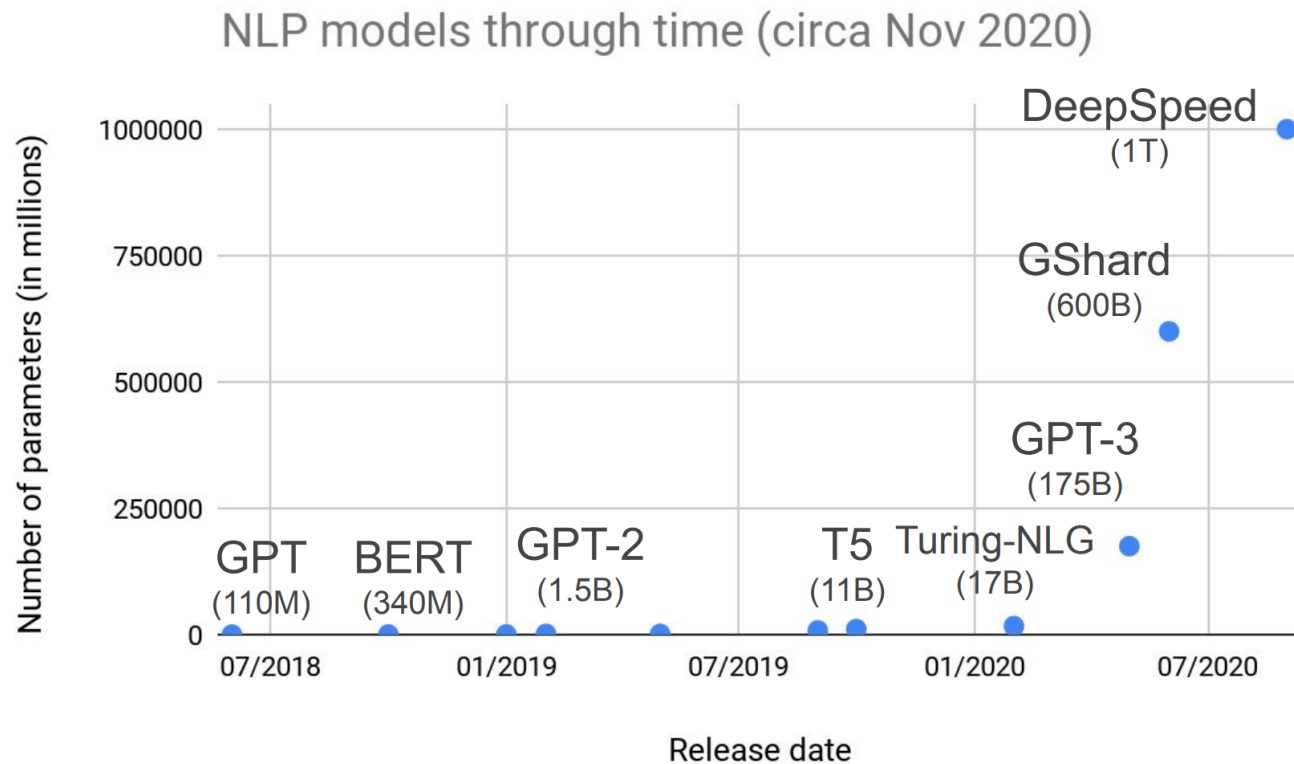
Table 1: Estimated CO₂ emissions from training common NLP models, compared to familiar consumption.¹



NLP 모델 하나 학습시키는데 도대체 얼마나 많은 에너지가 소비되는 걸까?

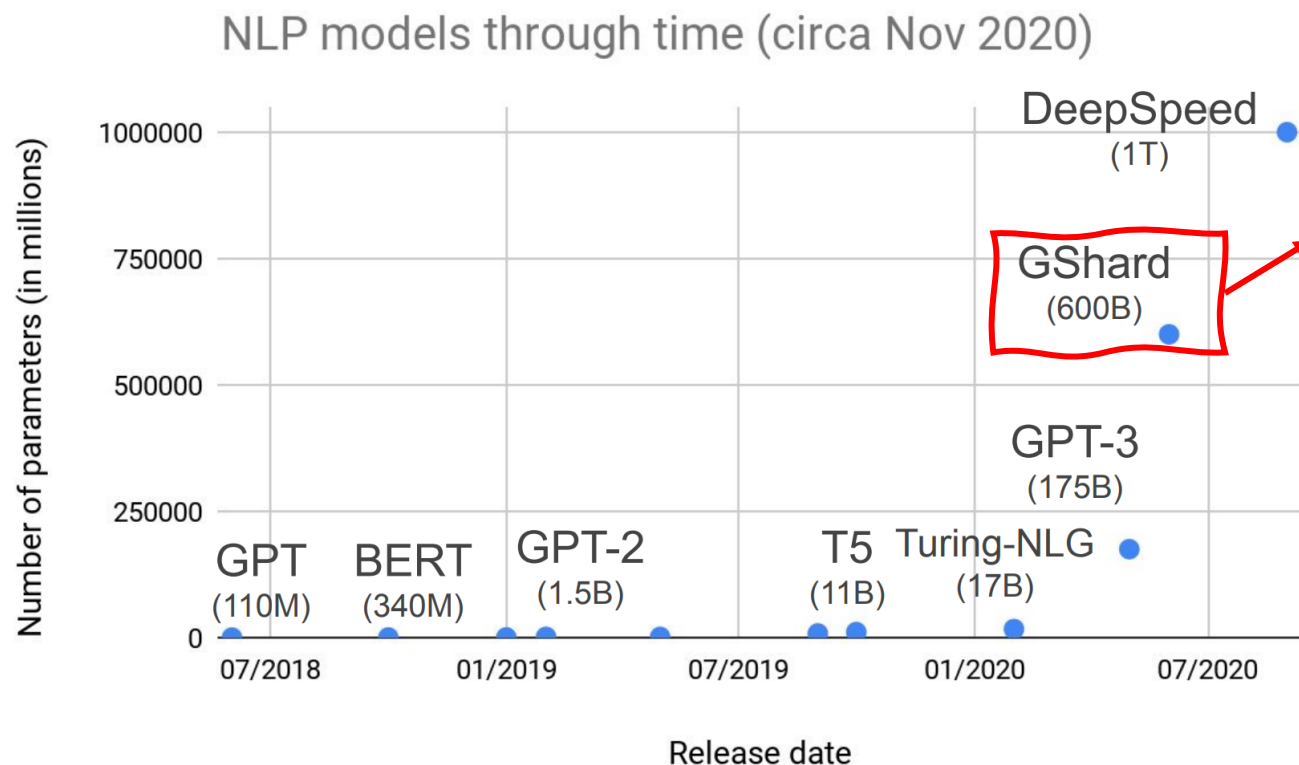
NAS 를 통해 Transformer model 하나 학습시키면 지구 온난화의 주범이...

Motivation



그럼에도 불구하고 NLP 모델 사이즈는...!

Motivation



Mixture of Expert (MoE) 기반의 sparsely-active transformers

MoE 는

- (1) complexity,
- (2) communication costs, and
- (3) training instabilities

위 세가지 문제로 널리 쓰이지는 못하고 있다!

위 세가지 문제를 해결한, 효율적인 sparsely-active transformers 를 만들어보자!!!!



What is the Mixture of Expert (MoE)?



Two Imagenet images from the same class

What is the Mixture of Expert (MoE)?

강아지 부분
인식전문가



배경 분리
전문가



객체 탐지
전문가

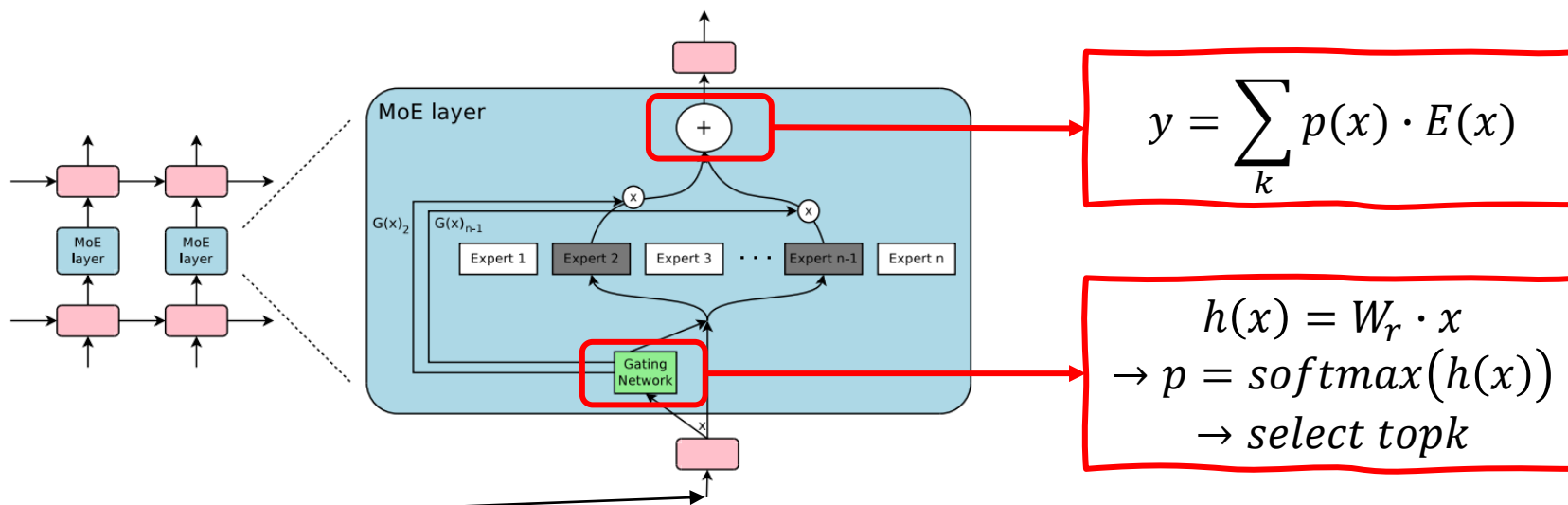


Logic
Control



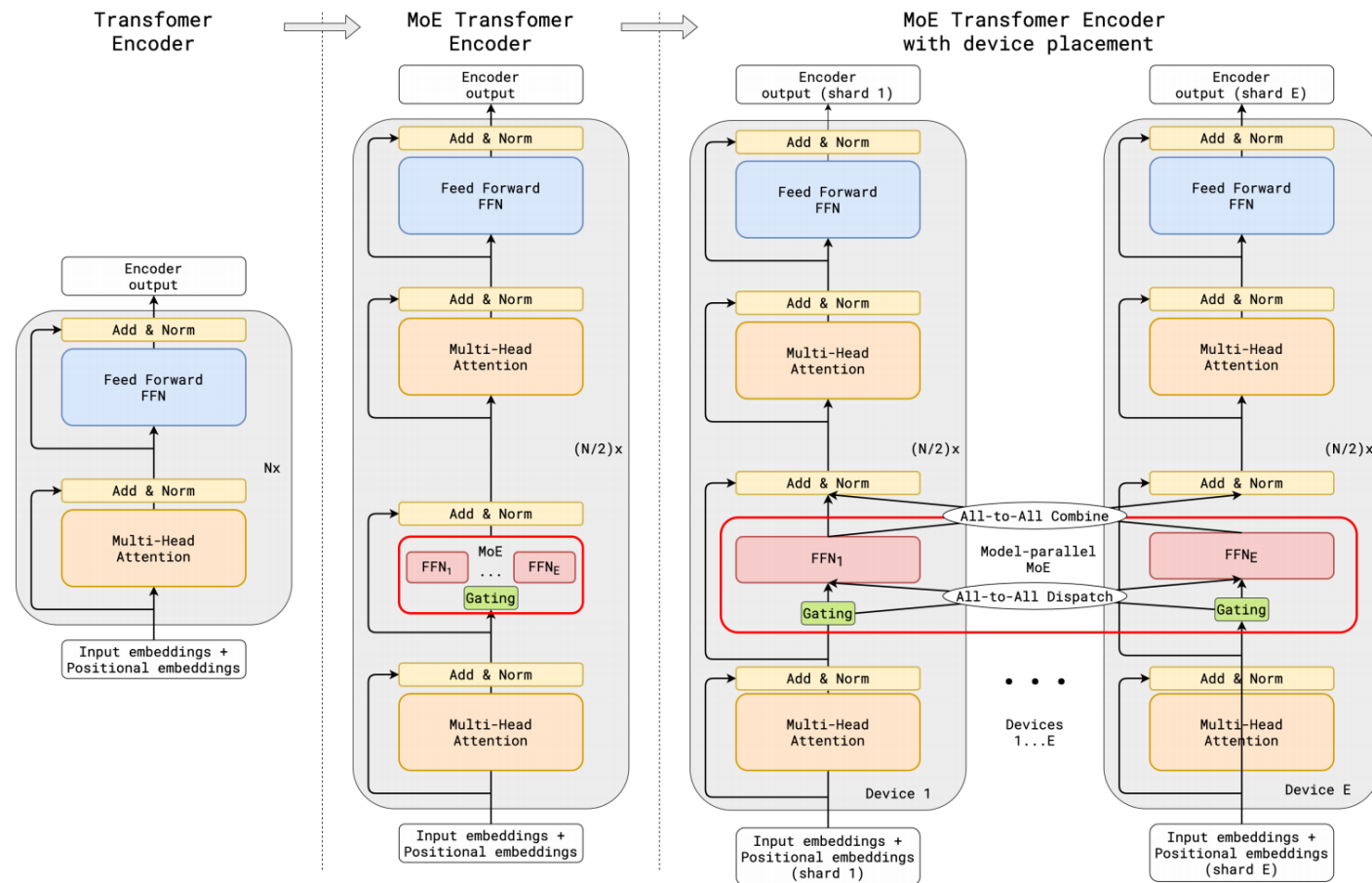
Two Imagenet images from the same class

MoE Layer



Two Imagenet images from the same class

MoE for Transformer



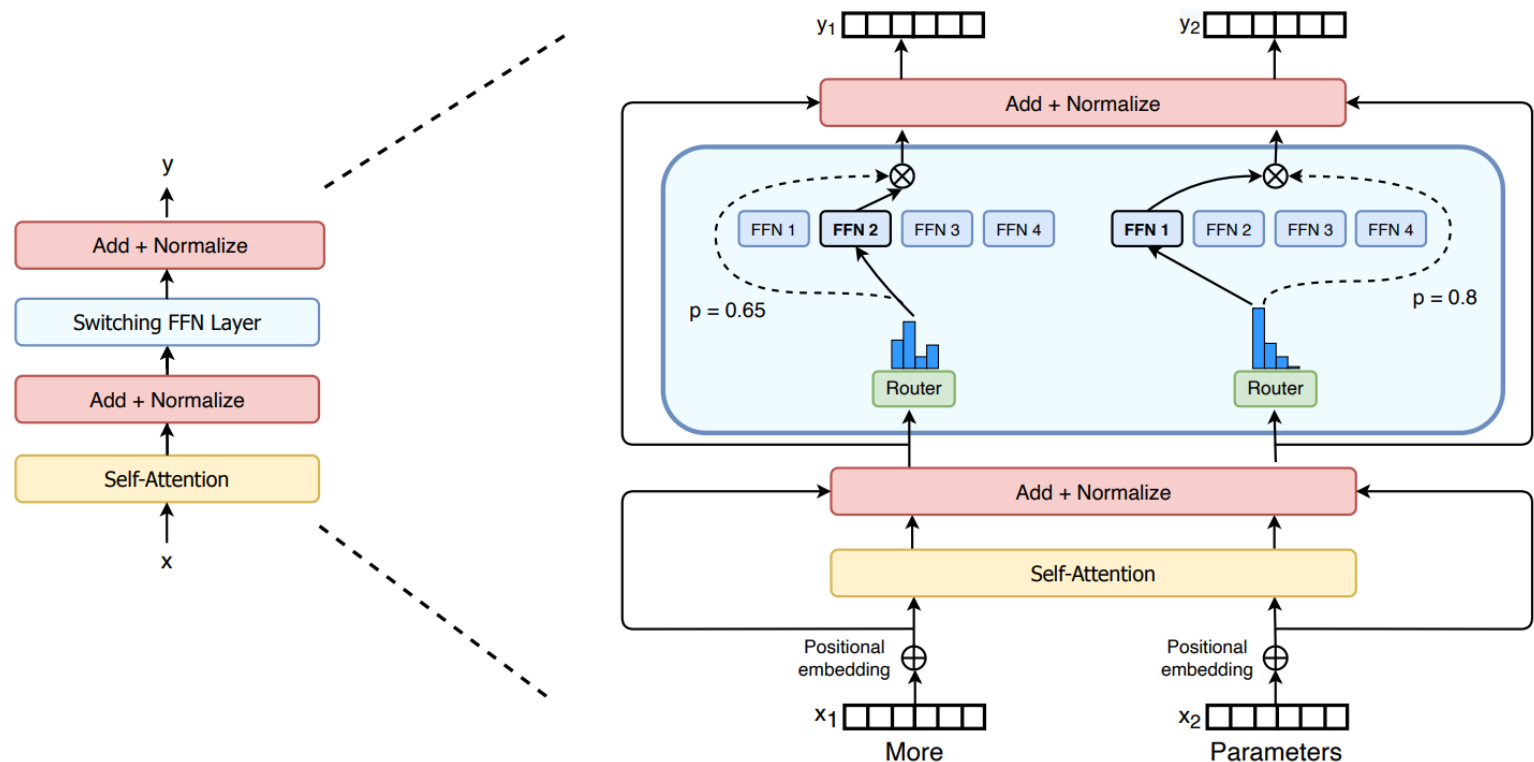
Transformer에는 feed forward network에만 MoE 적용

Routing은 token 단위로 적용



Any Question?

Basic idea for Switch Transformer



오직 하나의 expert 만 선택하자!

1. Single expert를 사용하여 Router 연산을 줄임! (top-k $\rightarrow$ top-1)
2. K개의 Expert 선택할 경우 데이터를 k 개로 복사해야 하지만 1개를 선택할 경우 그럴 필요가 없으므로 기존 MoE 기법에 비해 batch 사이즈가 줄어드는 효과
3. Routing 연산 후 Device 간의 communication 작업이 필요한데 1개의 expert 만을 선택함으로써 이를 줄일 수 있음

Logic Control

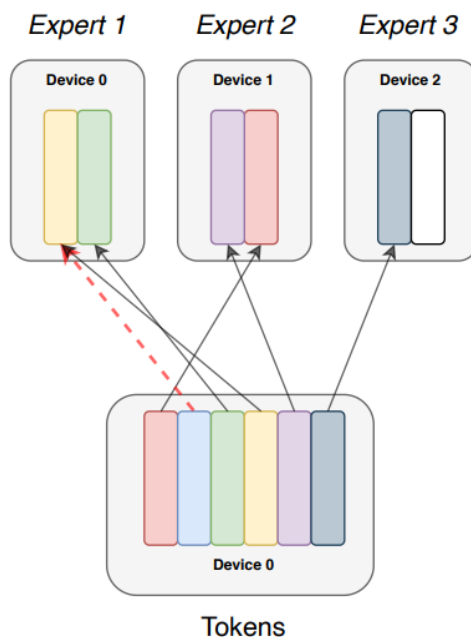
k = 1 routing strategy

(1) Distributed Switch Implementation.

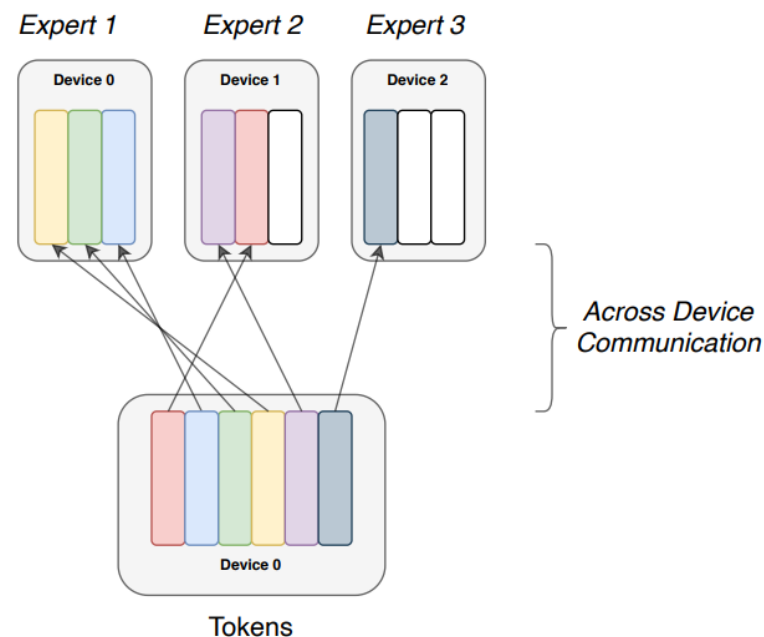
Terminology

- **Experts:** Split across devices, each having their own unique parameters. Perform standard feed-forward computation.
- **Expert Capacity:** Batch size of each expert. Calculated as $(\text{tokens_per_batch} / \text{num_experts}) * \text{capacity_factor}$
- **Capacity Factor:** Used when calculating expert capacity. Expert capacity allows more buffer to help mitigate token overflow during routing.

(Capacity Factor: 1.0)



(Capacity Factor: 1.5)



Logic Control

$$\text{expert capacity} = \frac{\text{tokens per batch}}{\text{number of experts}} \times \text{capacity factor}$$

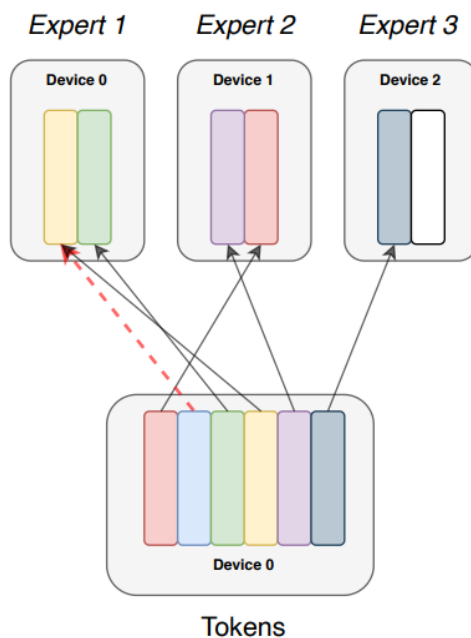
k = 1 routing strategy

(2) A Differentiable Load Balancing Loss.

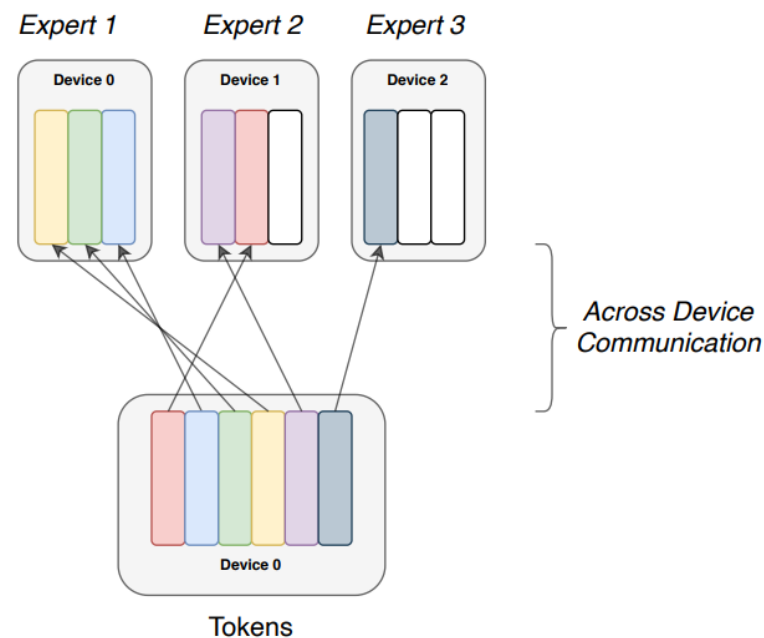
Terminology

- **Experts:** Split across devices, each having their own unique parameters. Perform standard feed-forward computation.
- **Expert Capacity:** Batch size of each expert. Calculated as
• $(\text{tokens_per_batch} / \text{num_experts}) * \text{capacity_factor}$
- **Capacity Factor:** Used when calculating expert capacity. Expert capacity allows more buffer to help mitigate token overflow during routing.

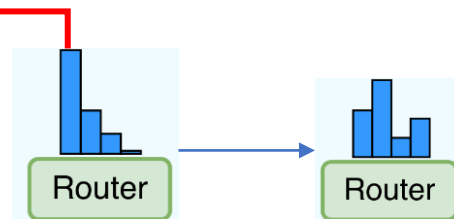
(Capacity Factor: 1.0)



(Capacity Factor: 1.5)



$$\text{loss} = \alpha N \cdot \sum_{i=1}^N f_i \cdot P_i$$



Logic Control

Benchmarking Switch versus MoE

a quality threshold of
Neg. Log Perp.=-1.495.



Model	Capacity Factor	Quality after 100k steps (Neg. Log Perp.)	Time to Quality Threshold (hours)	Speed (examples/sec)
T5-Base	—	-1.731	Not achieved [†]	1600
T5-Large	—	-1.550	131.1	470
MoE-Base	2.0	-1.547	68.7	840
Switch-Base	2.0	-1.554	72.8	860
MoE-Base	1.25	-1.559	80.7	790
Switch-Base	1.25	-1.553	65.0	910
MoE-Base	1.0	-1.572	80.1	860
Switch-Base	1.0	-1.561	62.8	1000
Switch-Base (expand)	1.0	-1.534	67.6	780

Improved Training and Fine-Tuning Techniques

(1) Selective Precision



Model (precision)	Quality (Neg. Log Perp.)	Speed (Examples/sec)
Switch-Base (float32)	-1.718	1160
Switch-Base (bfloat16)	-3.780 [<i>diverged</i>]	1390
Switch-Base (Selective precision)	-1.716	1390

Improved Training and Fine-Tuning Techniques

(2) Selective Dropout



Model (dropout)	GLUE	CNNDM	SQuAD	SuperGLUE
T5-Base (d=0.1)	82.9	19.6	83.5	72.4
Switch-Base (d=0.1)	84.7	19.1	83.7	73.0
Switch-Base (d=0.2)	84.4	19.2	83.9	73.2
Switch-Base (d=0.3)	83.9	19.6	83.4	70.7
Switch-Base (d=0.1, ed=0.4)	85.2	19.6	83.7	73.0

Expert dropout 과 기존 dropout의 비율을 적절히 조정해
줬을 때 최적의 성능이 나옴

Improved Training and Fine-Tuning Techniques

(3) A Better Initialization



Model (Initialization scale)	Average Quality (Neg. Log Perp.)	Std. Dev. of Quality (Neg. Log Perp.)
Switch-Base (0.1x-init)	-2.72	0.01
Switch-Base (1.0x-init)	-3.60	0.68

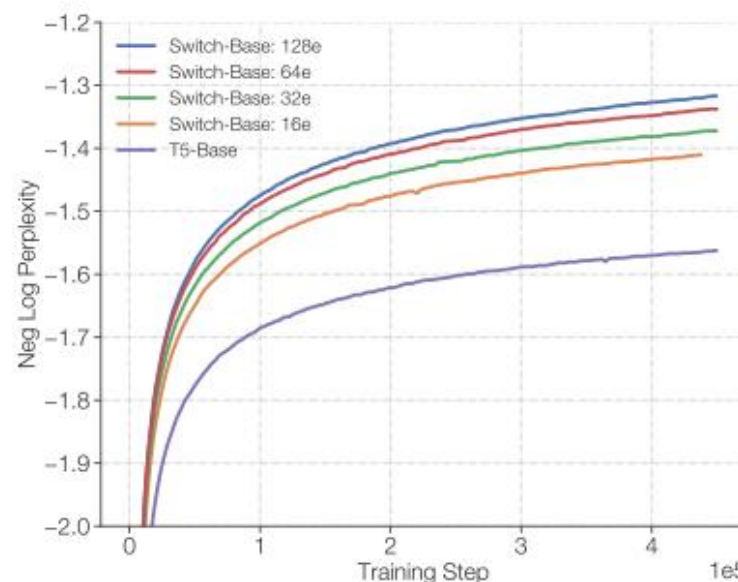
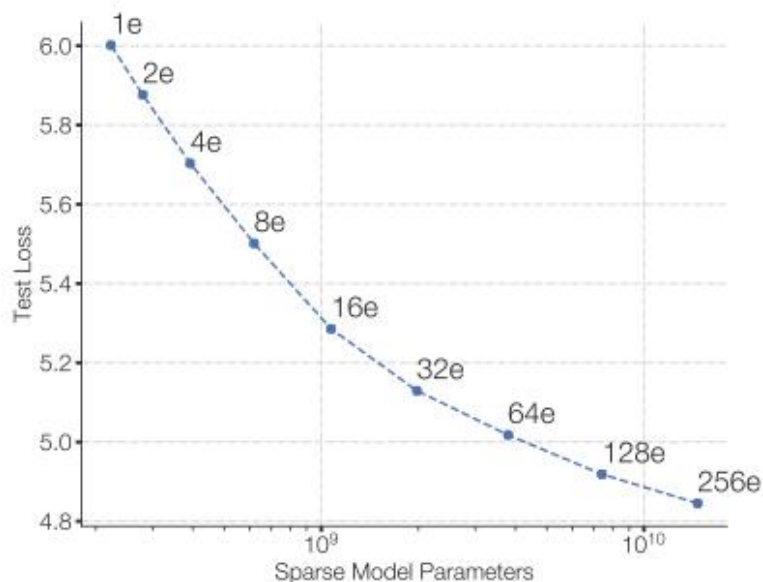
Truncated normal distribution 으로 초기화 해준 수에 0.1
을 곱했을 때 결과의 편차가 가장 적었음 → stable results



Any Question?

Scaling Properties

(1) Results on a step-basis (총 training step 고정)

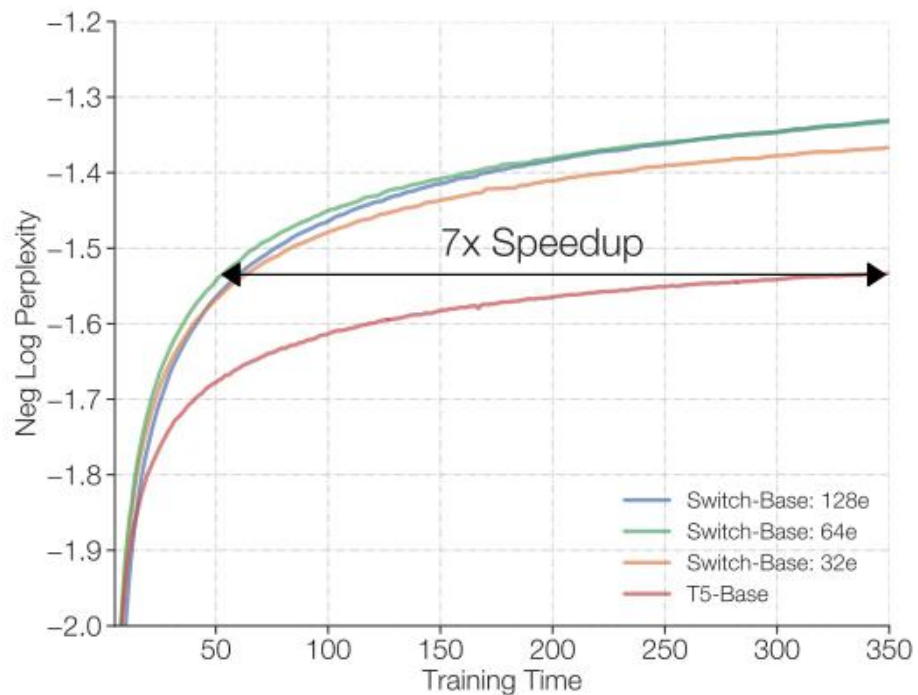


- FLOPS 등 다른 변수는 모두 동일한 상태에서 비교
- 결과 : (1) Experts 가 증가할 수록 성능 향상 (2) T5 base model 이 450K 스텝에서 낼 수 있는 성능을 60K 만에 도달

Logic Control

Scaling Properties

(2) Results on a time-basis (총 training time 고정)

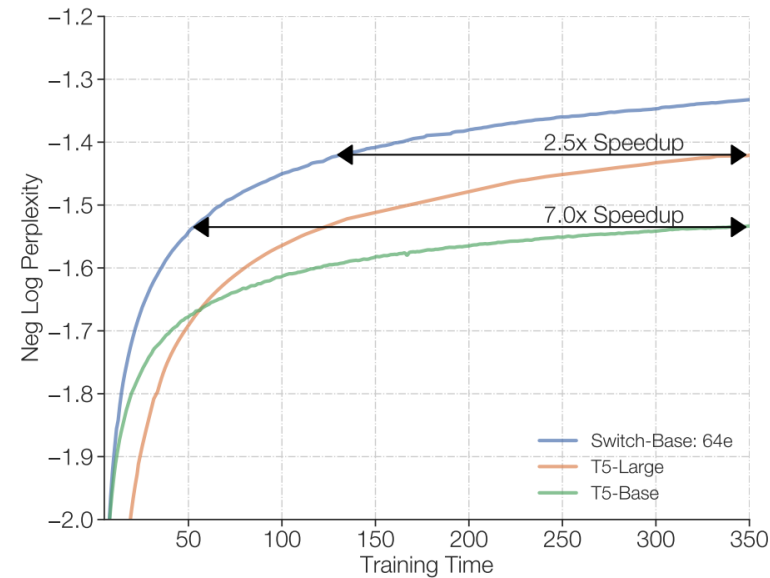
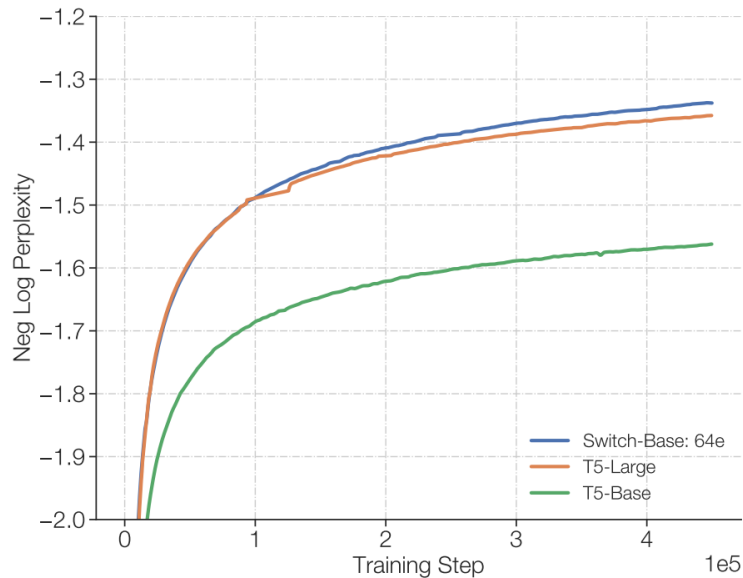


- 제한된 시간과 메모리 용량 하에서 T5 와 Switch 다시 비교
- 결과 : T5 보다 7 배 빨리 성능을 낼 수 있음

Logic Control

Scaling Properties

(3) Scaling vs A Large Dense Model (총 parameter 수 고정)



- T5-large 의 경우 각 토큰당 연산량이 3.5배 많음
- 결과1 : T5-base 보다 7배 빨리 성능을 낼 수 있음
- 결과2 : T5-large 보다 2.5배 빨리 성능을 낼 수 있음

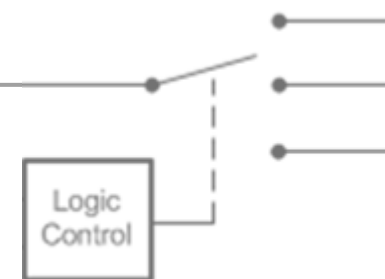




Any Question?

Fine-tuning

(1) Baseline and Switch models used for fine-tuning

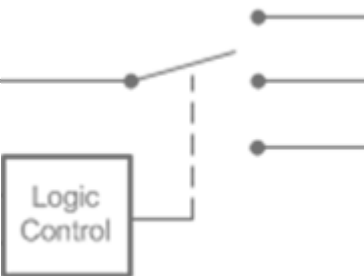


Model	Parameters	FLOPS
T5-Base	223M	124B
Switch-Base	7.4B	124B
T5-Large	739M	425B
Switch-Large	26.3B	425B

Table 5: **Comparing parameters and FLOPs of fine-tuning models.** Total number of parameters and floating point operations (FLOPs) per sequence for our models. In both cases, we match the computation of the corresponding T5 baseline because a single expert is always used per layer, but has many more parameters because each layer contains many experts.

Fine-tuning

(2) Fine-tuning tasks and datasets.



Model	GLUE	SQuAD	SuperGLUE	Winogrande (XL)
T5-Base	84.3	85.5	75.1	66.6
Switch-Base	86.7	87.2	79.5	73.3
T5-Large	87.8	88.1	82.7	79.1
Switch-Large	88.5	88.6	84.7	83.0

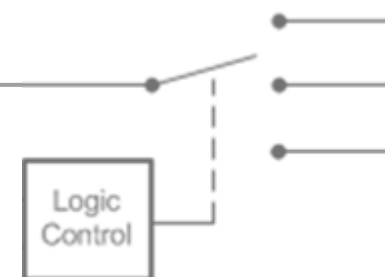
Model	XSum	ANLI (R3)	ARC Easy	ARC Chal.
T5-Base	18.7	51.8	56.7	35.5
Switch-Base	20.3	54.0	61.3	32.8
T5-Large	20.9	56.6	68.8	35.5
Switch-Large	22.3	58.6	66.0	35.5

Model	CB Web QA	CB Natural QA	CB Trivia QA
T5-Base	26.6	25.8	24.5
Switch-Base	27.4	26.8	30.7
T5-Large	27.7	27.6	29.5
Switch-Large	31.3	29.5	36.9

Table 6: **Fine-tuning results.** Fine-tuning results of T5 baselines and Switch models across a diverse set of natural language tests (validation sets; higher numbers are better). We compare FLOP-matched Switch models to the T5-Base and T5-Large baselines. For most tasks considered, we find significant improvements of the Switch-variants. We observe gains across both model sizes and across both reasoning and knowledge-heavy language tasks.

Distillation

(1) Distillation techniques..

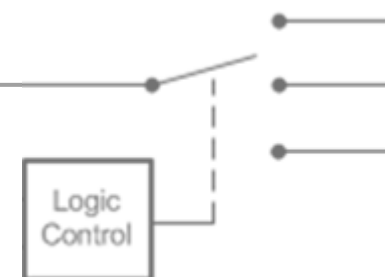


Technique	Parameters	Quality
T5-Base	223M	-1.636
Switch-Base	3,800M	-1.444
Distillation	223M	(3%) -1.631
+ Init. non-expert weights from teacher	223M	(20%) -1.598
+ 0.75 mix of hard and soft loss	223M	(29%) -1.580
Initialization Baseline (no distillation)		
Init. non-expert weights from teacher	223M	-1.639

Table 7: **Distilling Switch Transformers for Language Modeling.** Initializing T5-Base with the non-expert weights from Switch-Base and using a loss from a mixture of teacher and ground-truth labels obtains the best performance. We can distill 30% of the performance improvement of a large sparse model with 100x more parameters back into a small dense model. For a final baseline, we find no improvement of T5-Base initialized with the expert weights, but trained normally without distillation.

Distillation

(2) Achievable compression rates.

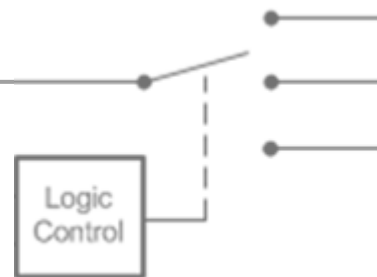


	Dense	Sparse				
Parameters	223M	1.1B	2.0B	3.8B	7.4B	14.7B
Pre-trained Neg. Log Perp.	-1.636	-1.505	-1.474	-1.444	-1.432	-1.427
Distilled Neg. Log Perp.	—	-1.587	-1.585	-1.579	-1.582	-1.578
Percent of Teacher Performance	—	37%	32%	30 %	27 %	28 %
Compression Percent	—	82 %	90 %	95 %	97 %	99 %

Table 8: **Distillation compression rates.** We measure the achievable quality while distilling a sweep of sparse models into a small dense baseline. Our dense baseline, T5-Base, has a -1.636 Neg. Log Perp.. In the right columns, we distill increasingly large sparse models into the same T5-Base architecture. Through a combination of weight-initialization and a mixture of hard and soft losses, we can shrink our sparse teachers by 95%+ while preserving 30% of the quality gain.

Distillation

(3) Distilling a fine-tuned model.



Model	Parameters	FLOPS	SuperGLUE
T5-Base	223M	124B	74.6
Switch-Base	7410M	124B	81.3
Distilled T5-Base	223M	124B	(30%) 76.6

Table 9: **Distilling a fine-tuned SuperGLUE model.** We distill a Switch-Base model fine-tuned on the SuperGLUE tasks into a T5-Base model. We observe that on smaller datasets our large sparse model can be an effective teacher for distillation. We find that we again achieve 30% of the teacher’s performance on a 97% compressed model.

Multilingual Learning

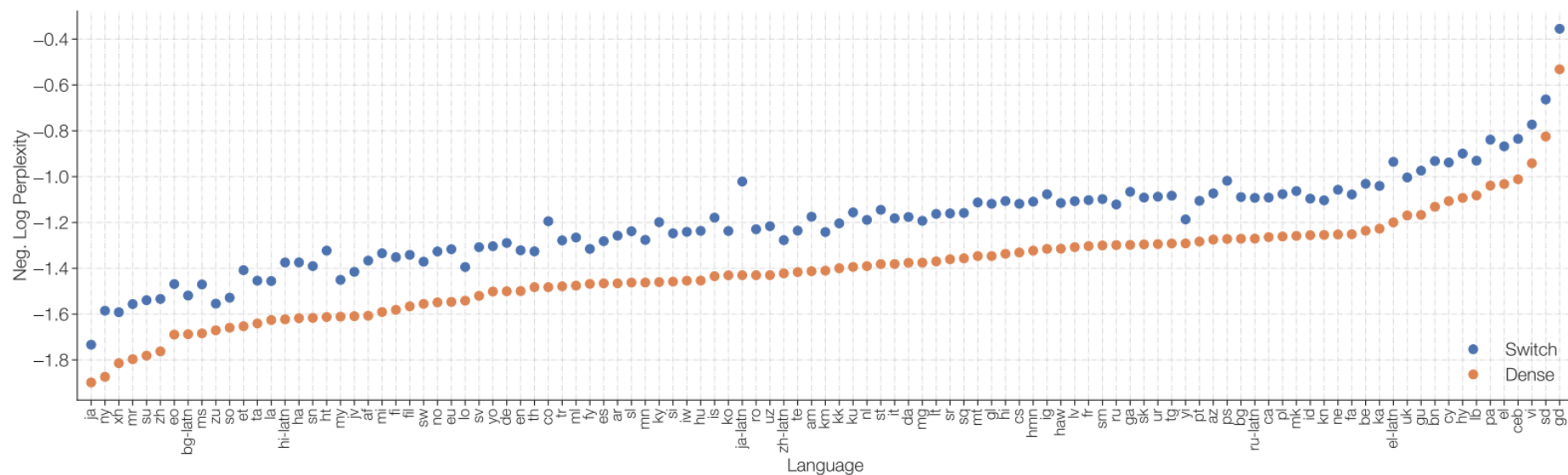
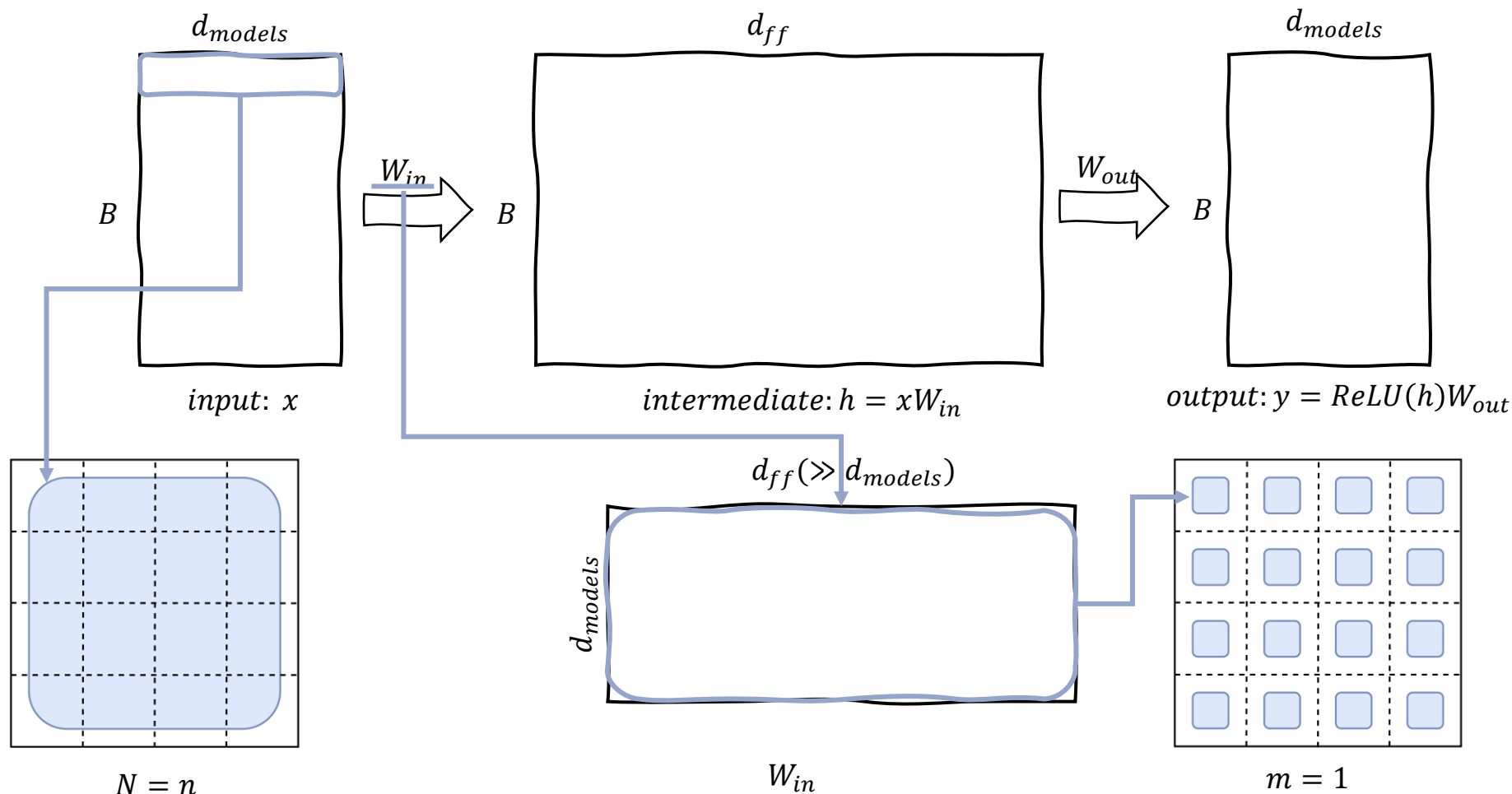


Figure 7: **Multilingual pre-training on 101 languages.** Improvements of Switch T5 Base model over dense baseline when multi-task training on 101 languages. We observe Switch Transformers to do quite well in the multi-task training setup and yield improvements on all 101 languages.

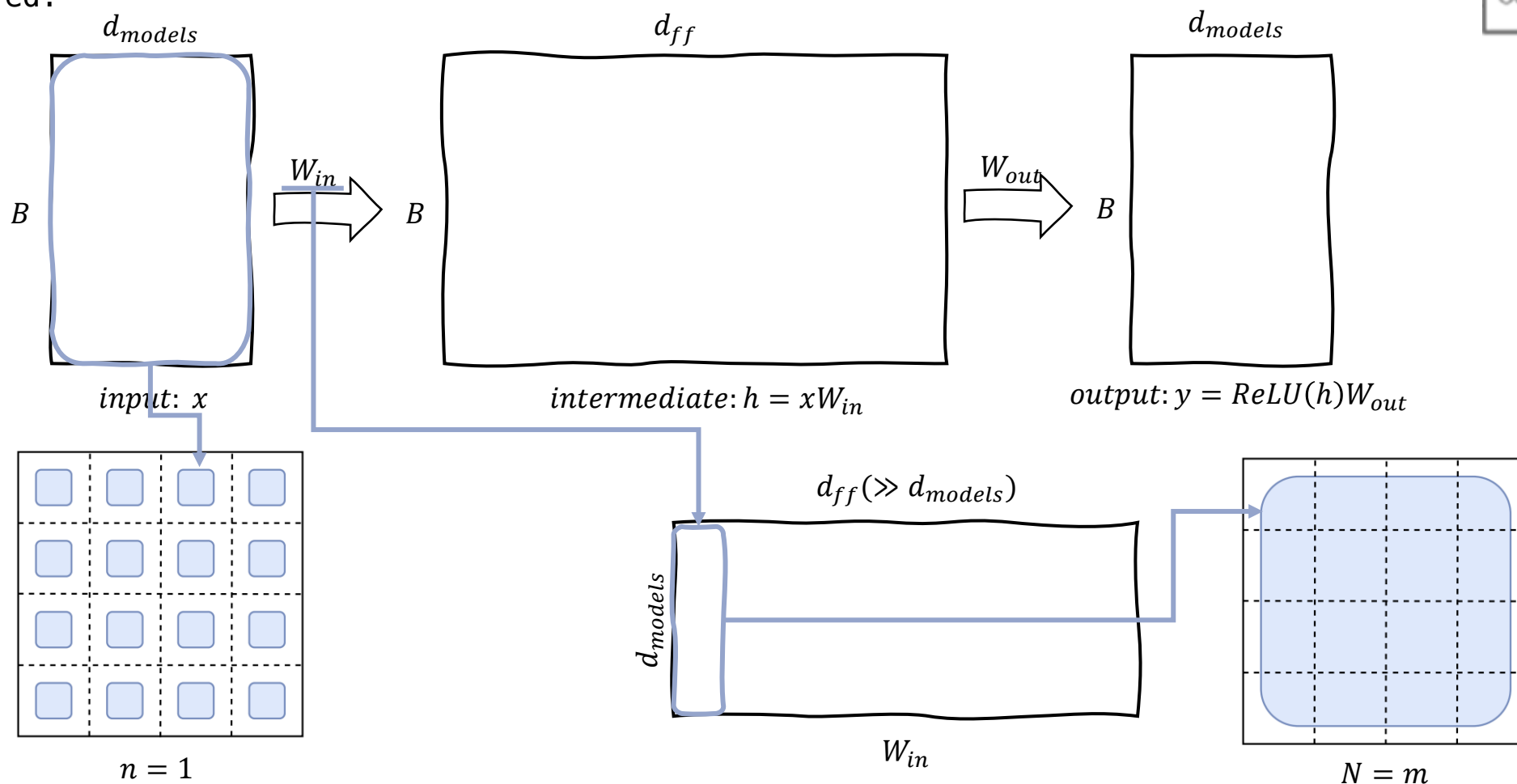
Designing models with data, model, and expert-parallelism

Data parallelism: This has the advantage that no communication is needed until the entire forward and backward pass is finished and the gradients need to be then aggregated across all cores



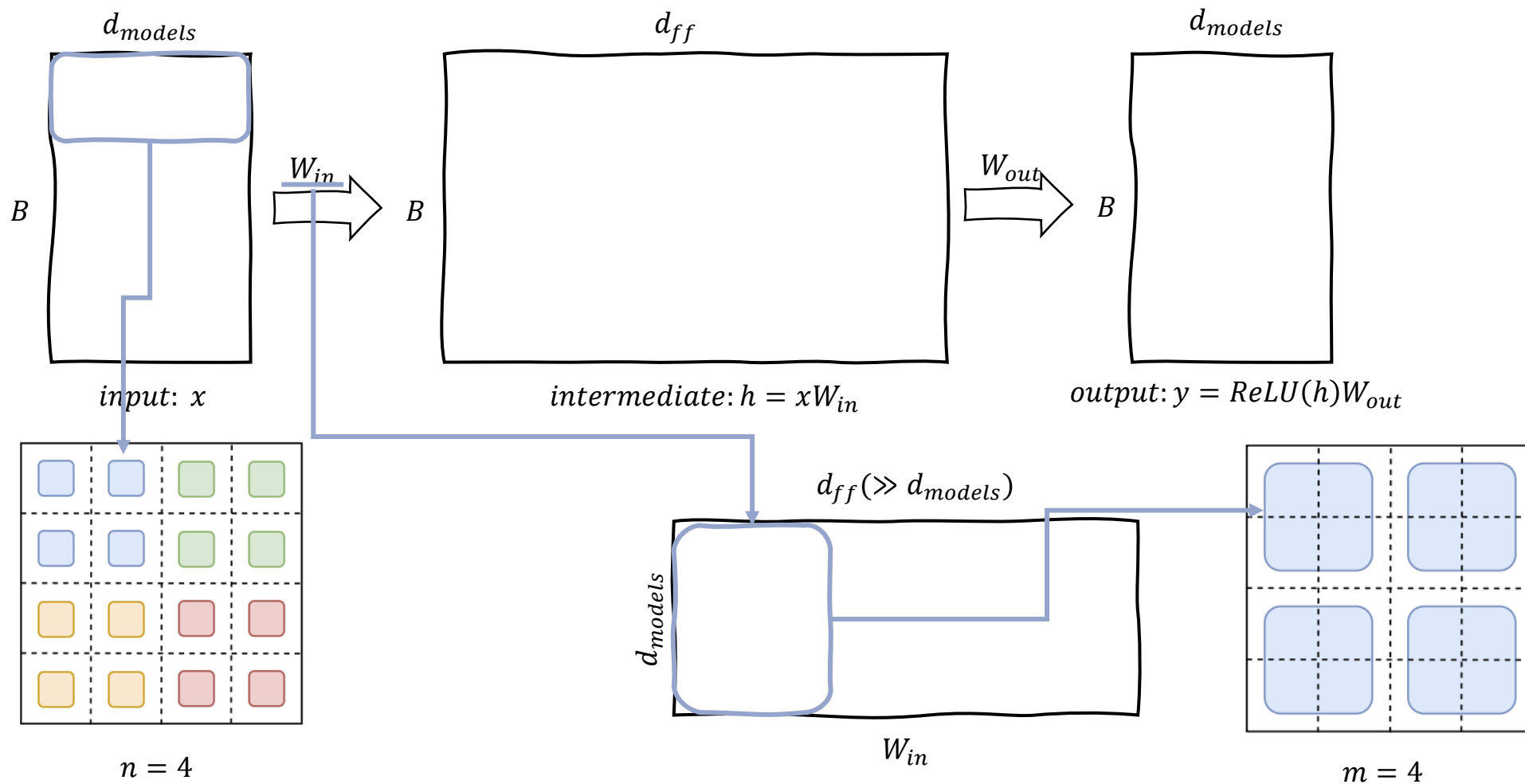
Designing models with data, model, and expert-parallelism

Model parallelism: All cores must keep the full B tokens and each core will contain a unique slice of the weights. For each forward and backward pass, a communication cost is now incurred.



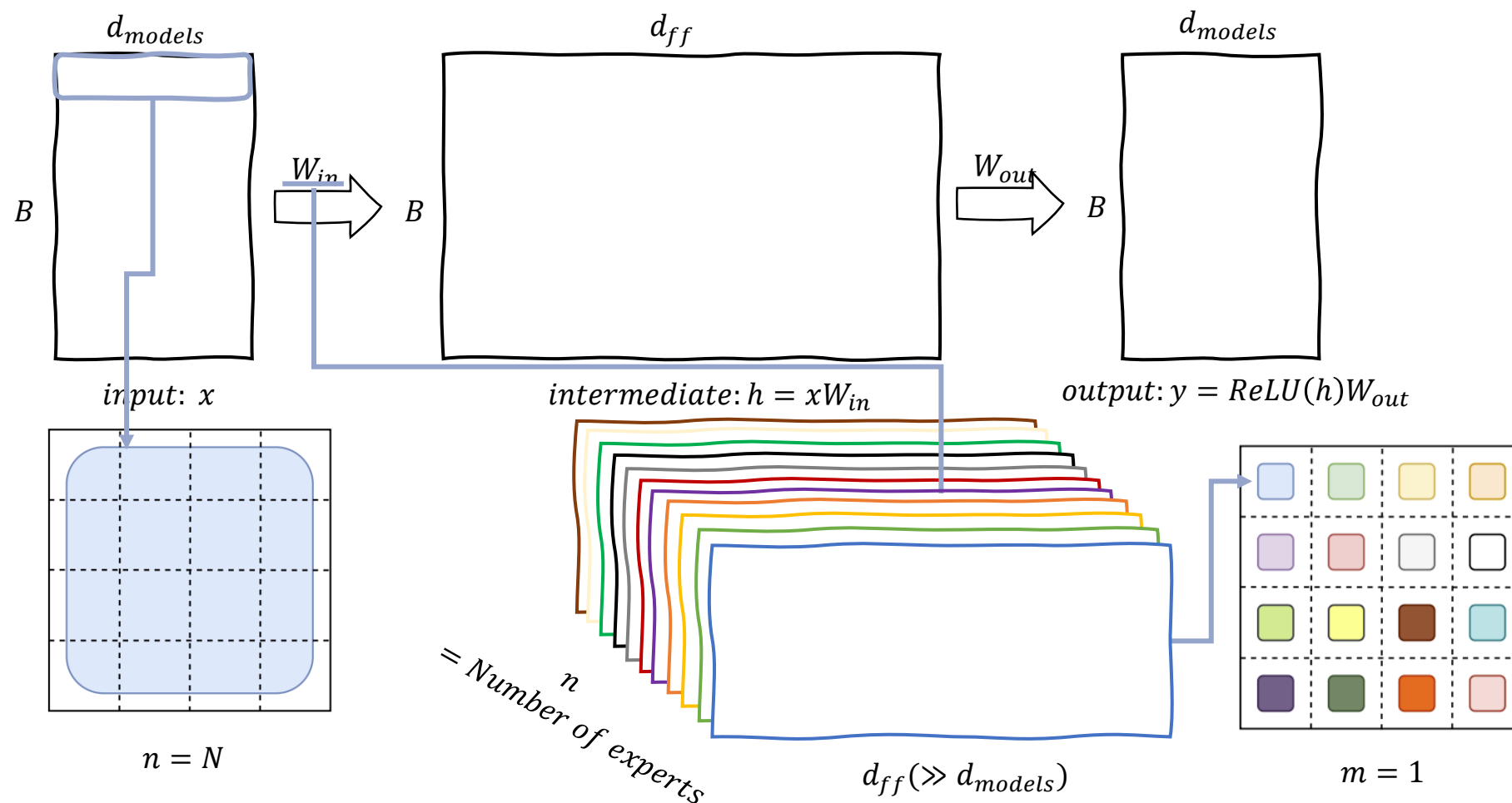
Designing models with data, model, and expert-parallelism

Model And Data parallelism: Each core will be responsible for B/n tokens and d_{ff}/m of both the weights and intermediate activation.



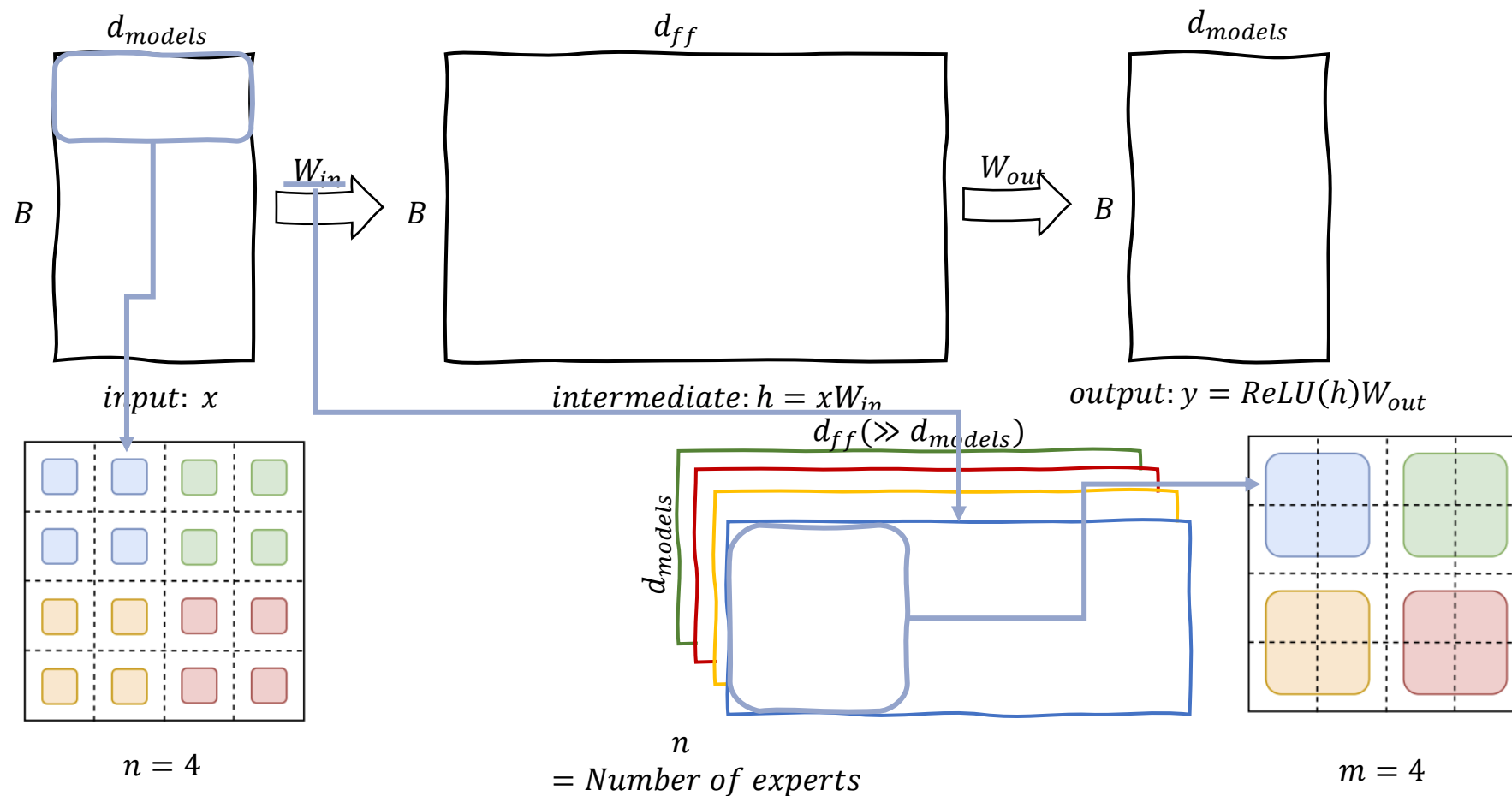
Designing models with data, model, and expert-parallelism

Expert And Data parallelism: Each core will be responsible for B/n tokens and d_{ff}/m of both the weights and intermediate activation.



Designing models with data, model, and expert-parallelism

Expert, Data and Model parallelism: Each core will be responsible for B/n tokens and d_{ff}/m of both the weights and intermediate activation.



Designing models with data, model, and expert-parallelism



Model	Parameters	FLOPs/seq	d_{model}	FFN_{GEGLU}	d_{ff}	d_{kv}
T5-XXL	13B	8.7T	4096	✓	10240	64
Switch-XXL	395B	8.7T	4096	✓	10240	64
Switch-C	1571B	890B	2080		6144	64

Model	Num. Heads	Num. Layers	Num. Experts	Neg. Log Perp. @250k	Neg. Log Perp. @ 500k
T5-XXL	128	24	—	-1.147	-1.095
Switch-XXL	128	24	64	-1.086	-1.008
Switch-C	32	15	2048	-1.096	-1.043

Sample efficiency versus T5-XXL: The gap continues to increase with additional training, with the Switch-XXL model out-performing the T5-XXL by 0.087 by 500k steps.

Training instability: We find that the larger Switch-C model, with 1.6T parameters and 2048 experts, exhibits no training instability at all. Instead, the Switch XXL version, with nearly 10x larger FLOPs per sequence, is sometimes unstable

DISCUSSION

- **Isn't Switch Transformer better due to sheer parameter count?**

Yes, and by design! Parameters, independent of the total FLOPs used, are a useful axis to scale neural language models.

- **I don't have access to a supercomputer – is this still useful for me?**

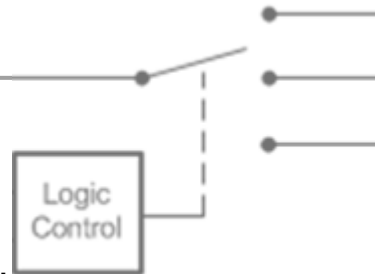
Though this work has focused on extremely large models, we also find that models with as few as two experts improves performance while easily fitting within memory constraints of commonly available GPUs or TPUs

- **Do sparse models outperform dense models on the speed-accuracy pareto curve?**

Yes. Across a wide variety of different model's sizes, sparse models outperform dense models per step and on wall clock time.

- **I can't deploy a trillion parameters model – can we shrink these models?**

We cannot fully preserve the model quality, but compression rates of 10 to 100x are achievable by distilling our sparse models into dense models while achieving $\approx 30\%$ of the quality gain of the expert model.





Any Question?